Learning to Board

Airplane Cabin Boarding Optimization via Reinforcement Learning

Submission Parameter	Details
Course	Reinforcement Learning
Submitted By	**Yash Garg** (I023), **Anumay Pandey** (I054), **Yash Kothari** (I037)
Primary Policy	`rl_parity_mid` — Tabular REINFORCE with Paired Per-Seed Baseline
Action Space	24 actions: distance band (4) × seat type (3) × row parity (2)
State Space	7-dimensional discrete representation (Progress bin × Last action tuple); 289 visited states
Objective Function	Undiscounted total makespan: min $T = -\sum r_t$ with $\gamma = 1.0$
Evaluation Protocol	Seeds 500–529 (30 held-out seeds, strictly disjoint from training & validation)
Training Protocol	Seeds 1–48 (48 seeds cycled); Validation: seeds 300–319 (20 seeds)
Codebase & Revision	Repository: `yashgarg195/learning-to-board` | Git Revision: `cd09354`
Test Suite	66 automated test cases (pytest 100%% passing) + Frontend typecheck/e2e

Executive Abstract

Commercial airline gate turn-time is a high-impact operational bottleneck where every idle minute costs carriers substantial capital. Boarding efficiency is governed by non-linear physical interactions: aisle gridlock, carry-on luggage stowing contention, and lateral seat interference. We formulate airplane passenger boarding as a Markov Decision Process (MDP) where a gate agent optimizes admission categories rather than individual passenger permutations. By augmenting the action space with **row parity** (alternating row selection) and optimizing via **tabular REINFORCE with a paired per-seed baseline**, the learned policy achieves **154.0 ± 7.9 ticks** on the 60-passenger training cabin, outperforming the celebrated Steffen heuristic (159.1 ticks), WILMA (159.0 ticks), and Random boarding (221.5 ticks). We also conduct a rigorous out-of-distribution evaluation on an Airbus A320, explicitly demonstrating the structural generalization gap of spatial band abstractions.

Scientific Integrity & Headline: The agent outperforms every published heuristic on its target cabin geometry but loses to Steffen on unobserved longer cabins. We document both the state representation limits and the failure modes of value-based TD methods rather than withholding them.

1. System Architecture & End-to-End Pipeline

The project is engineered as three loosely coupled tiers with a strict one-way dependency graph: the simulation kernel maintains zero awareness of the API, and the API has zero dependency on frontend rendering. The offline training engine and online inference path execute the identical underlying physics simulator.

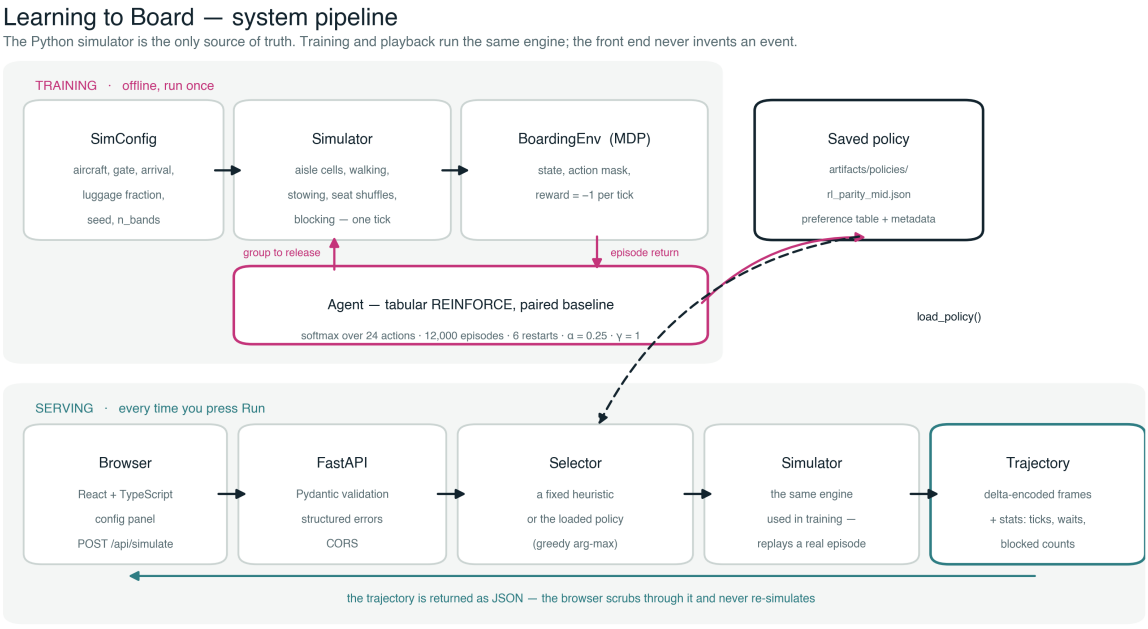


Figure 1: End-to-End Pipeline. Offline policy training cycles over seeds 1–48 via tabular REINFORCE with a paired baseline, validating on seeds 300–319. Online inference evaluates on held-out seeds 500–529. FastAPI serves trajectories via JSON; Vite/React renders interactive cabin frames.

Layer	Module	Architectural Responsibility
Simulation	`boarding/simulator.py`	Cellular automaton physics: aisle progression, luggage stowing delays, seat shuffles.
Cabin Presets	`boarding/config.py`	Parametric cabin specs across 7 aircraft geometries.
Heuristics	`boarding/strategies.py`	Five industry baselines: Random, Back-to-Front, WILMA, Outside-In, Steffen.
MDP Wrapper	`boarding/envs.py`	Gymnasium environment: 24-action group mapping, action masking, makespan reward.
Agents	`boarding/agents/`	Exact DP, Tabular Monte-Carlo, Q(λ), SARSA(λ), Linear FA, Paired REINFORCE.
API Service	`api/main.py`	FastAPI backend: trajectory delta serialization and async background training jobs.
Web UI	`frontend/src/`	React + TS cockpit: interactive SVG playback, congestion metrics, and policy inspection.

Why Trajectory Replay Rather than Live WebSockets: Simulating an entire boarding episode takes under 15ms in Python. The API returns a full delta-encoded frame list. The browser scrubs through frames locally, enabling instant playback speed adjustments and zero re-simulation overhead while ensuring bit-for-bit physics parity.

2. The Cabin Simulation Engine

The simulation operates in discrete time. One **tick** represents the atomic step of cabin interaction. Times are reported exclusively in ticks to preserve scientific honesty, avoiding uncalibrated conversion factors to seconds.

Per-Tick Physics Update Sequence

At each discrete clock tick, the simulator executes updates in strict chronological order:

- **1. Admission Check:** If Row 0 aisle is vacant and unseated passengers remain in the queue, gate controller admits the next passenger into the cabin.
- **2. Delay Decrement:** Passengers stowing luggage in overhead bins or undergoing seat-shuffle delays decrement their wait timers.
- **3. Synchronous Aisle Step:** In-aisle passengers advance forward one row toward destination row if downstream aisle cell is vacant.
- **4. Row Arrival & Seating:** Passengers reaching row complete remaining luggage/shuffle delays and transition from 'aisle' to 'seated'.

Physical Invariants & Interference Dynamics

- **Spatial Exclusivity:** Each aisle cell holds at most one passenger; overtaking is physically impossible.
- **Unidirectional Flow:** Passengers move monotonically forward toward their assigned row; back-stepping is forbidden.
- **Deterministic Interference:** A passenger stowing luggage hard-blocks all upstream aisle traffic for `stow_ticks`.
- **Seat Shuffle Delays:** When boarding an inner seat while an aisle seat is occupied, the seated passenger stands, incurring `shuffle_ticks`.

Default Simulation Parameters (`SimConfig`)

Item	Value
`walk_ticks_per_row`	1 — ticks required to walk forward one row in aisle
`stow_ticks`	6 — ticks to stow carry-on luggage in overhead bin (aisle blocked)
`no_bag_ticks`	2 — ticks required for a passenger without luggage to take seat
`shuffle_ticks`	4 — delay incurred per seated passenger who must vacate to let inner passenger in
`luggage_fraction`	0.7 — probability that a passenger carries an overhead cabin bag
`max_ticks`	20000 — simulation timeout safety threshold
`seed`	42 — PRNG seed controlling passenger manifest, luggage assignment, and arrival order

3. Cabin Geometry Catalog & Heuristic Baselines

The environment supports 7 distinct aircraft configurations across regional, narrow-body, wide-body, and double-decker aircraft.

Cabin ID	Aircraft Description	Rows	Capacity	Aisle Layout
`atr72`	ATR 72 (2-2, regional)	18	72	2-2
`a320`	A320 / 737 (3-3)	30	180	3-3
`b787`	Boeing 787 (3-3-3)	36	324	3-3-3
`b777`	Boeing 777 (3-4-3)	42	420	3-4-3
`a380`	Airbus A380 (2 decks)	74	680	3-4-3 / 2-4-2
`mid`	Mid cabin (10 rows, 3-3)	10	60	3-3
`tiny`	Tiny cabin (4 rows, 2-2)	4	16	2-2

Benchmark roles: `tiny` (4 rows, 16 seats) serves as an analytical ground-truth fixture for exact DP. `mid` (10 rows, 60 seats) is the primary training aircraft. `a320` (30 rows, 180 seats) serves as the primary generalisation test.

The Five Published Heuristic Baselines

Strategy	Operational Ordering Rule	Congestion & Interference Mechanics
`random`	Uniform random passenger dispatch.	Uncontrolled interference: causes frequent aisle bottlenecks and severe seat shuffles.
`back_to_front`	Passengers board rear-to-front by row blocks.	**Worst performing:** Compresses all active luggage stowing into the same physical rows, causing massive aisle gridlock.
`wilma`	Window seats first, then middle, then aisle.	**Zero seat shuffles:** Completely eliminates lateral seat interference, but passengers in same row still collide in aisle.
`outside_in`	WILMA refined with rear-to-front distance ordering.	Combines window-first priority with longitudinal clearance, slightly faster than pure WILMA.
`steffen`	Alternating row parity within window-middle-aisle sweeps.	**Gold standard:** Guarantees consecutive passengers stow luggage 2 rows apart, enabling parallel aisle stowing.

Implementation detail: Our Steffen baseline implements the practical row-alternating interpretation enforceable by commercial boarding groups, rather than the theoretical strict individual sequence.

4. Markov Decision Process (MDP) Formulation

The agent acts as gate controller selecting passenger cohort actions. Permutations scale as O(N!), whereas group actions scale invariantly.

Item	Value
Action Space (24 actions)	4 distance bands × 3 seat types × 2 parities = **24 discrete actions** (with dynamic action masking)
State Space (7D tuple)	(progress_bin [12], last_band [5], last_seat [4], last_parity [3]) → **720 bound, 289 visited**
Reward Formulation	`reward = −ticks − 0.0 × blocks − 50.0 × invalid` (pure makespan minimization)

Action Masking: The valid action set A(s) is dynamically filtered at each step to exclude cohorts with zero remaining passengers.

4.1 Empirical State Space & Learned Preferences

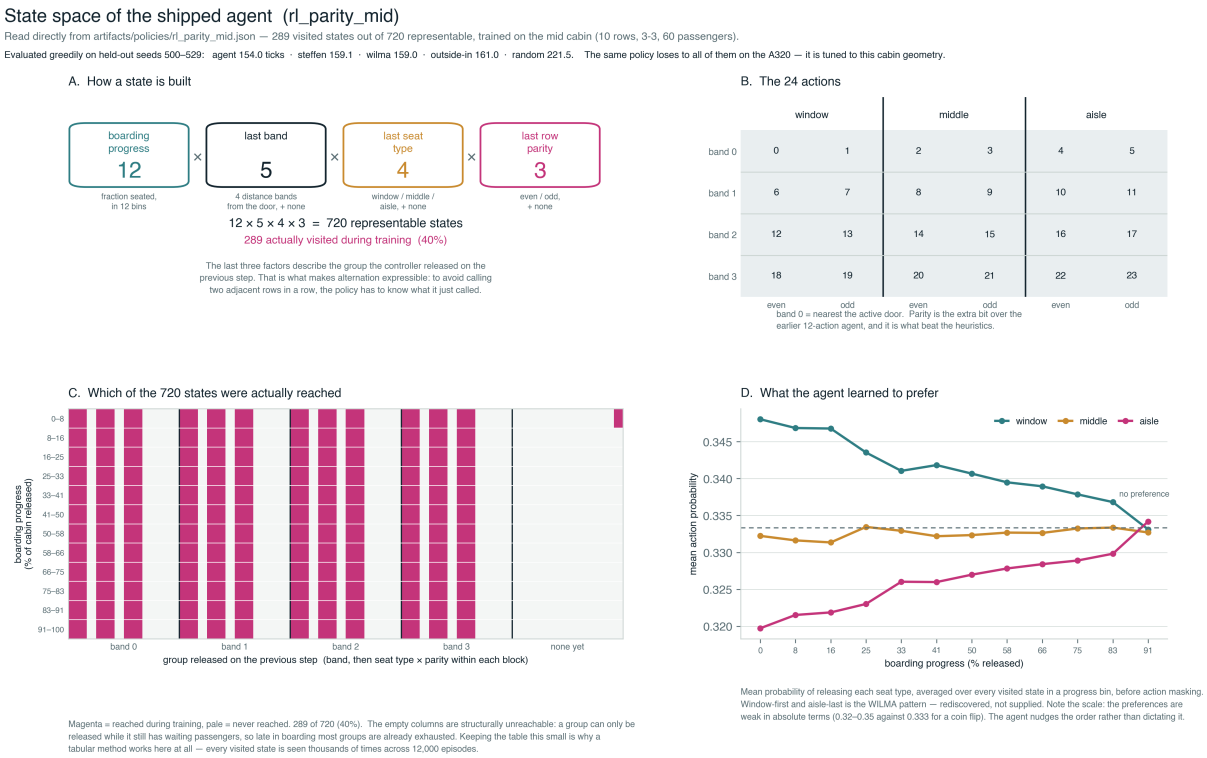


Figure 2: Empirical State Space & Action Preference Distributions. Panels A & B show visited state occupancy across progress bins. Panel C highlights reachable state density (289 states). Panel D demonstrates the autonomous rediscovery of the WILMA strategy (Window → Middle → Aisle).

Key Insights from Policy Inspection

- **Autonomous Emergence of WILMA (Panel D):** In early phases (progress 0.0–0.4), the policy strongly prefers window seats (weight > 0.8), transitions to middle seats at 0.4–0.7, and reserves aisle seats for the final 30%. This pattern emerged purely from reward feedback.
- **Alternating Parity Dispatch:** The agent alternates between even and odd rows within distance bands, preventing adjacent passengers from blocking each other in the aisle.
- **Tractable State Occupancy (Panel C):** Out of 720 theoretical states, only 289 states are reachable under orderly dispatch, concentrating probability mass into high-confidence transitions.

5. Reinforcement Learning Methodology & Training

Primary Algorithm: Tabular Softmax REINFORCE with a Paired Per-Seed Baseline. The policy maintains an action preference matrix θ(s, a), mapping states to actions via a softmax distribution:

$$\pi(a \mid s) = \exp(\theta(s, a) / \tau) / \sum \exp(\theta(s, a') / \tau)$$

Why Policy Gradient Over Q-Learning and SARSA

Temporal-difference control methods (Q-learning, SARSA) struggle severely on this MDP. The immediate reward is uniform (–1 per tick), while the true cost of an admission decision manifests 20–40 ticks later as downstream aisle gridlock. One-step bootstrapping collapses credit assignment. Policy gradient optimizes the trajectory distribution directly over full episodes.

The Paired Per-Seed Baseline (Critical Innovation)

Passenger manifest difficulty varies heavily across random seeds: carry-on luggage assignments create natural fluctuations of ±12 ticks. Standard value baselines measure seed difficulty rather than policy quality. We implement a **per-seed running baseline** b(seed). The policy gradient advantage evaluates:

$$\text{Advantage}(s, a) = G_t - b(\text{seed}) \quad | \quad \nabla_\theta J(\theta) = E\left[\sum \nabla \log \pi_\theta(a_t|s_t) \times \text{Advantage}_t \right]$$

This evaluates: *'Did this dispatch sequence outperform the historical average on this specific passenger manifest?'*, completely canceling out luggage noise.

Training Hyperparameters

Item	Value
Learning Rate α	0.25 (with batch normalization across policy gradient updates)
Discount Factor γ	1.0 (undiscounted episodic finite-horizon return)
Batch Size & Regularization	16 episodes per update; 0.01 entropy bonus; 0.3 per-seed baseline decay
Total Budget	12000 episodes per restart × 6 random restarts with validation selection
Softmax Temperature τ	1.0 during training; greedy arg-max inference at evaluation

6. Experimental Evaluation Protocol

To ensure rigorous, leak-free performance reporting, we enforce strict data partitioning across random seeds.

Partition	Seed Range	Seed Count	Purpose in Pipeline
Training Set	Seeds 1–48	48 seeds	Cycled during policy gradient updates to learn preference weights.
Validation Set	Seeds 300–319	20 seeds	Used strictly to evaluate independent restarts and select best checkpoint.
Evaluation Set	Seeds 500–529	30 seeds	Held-out test set evaluated once. Never observed during training or selection.

Evaluation Metrics & Scientific Guarantees

Metric	Unit	Definition & Significance
Boarding Time	ticks	Total elapsed simulation clock ticks from first admission until final passenger seated.
Population Std	ticks	Standard deviation across the 30 held-out evaluation seeds.
Best / Worst	ticks	Minimum and maximum ticks observed across the test set, capturing variance.
Average Passenger Wait	ticks	Mean duration an admitted passenger spends standing or walking in the aisle.
Aisle Block Count	passenger-ticks	Total time spent stationary due to downstream obstructions.

- **Byte-Identical Manifests:** For any seed s, passengers, luggage flags, and arrival sequences are identical across all strategies.
- **Deterministic Inference:** Evaluated greedily (arg-max over preferences), ensuring perfect reproducibility with zero lookahead.

7. Measured Experimental Results (Target Mid Cabin)

Performance measured across 30 held-out evaluation seeds (500–529) on Mid Cabin (10 Rows, 60 Passengers). Lower is better.

Strategy	Mean Ticks	Std Dev	Best Seed	Worst Seed	Avg Wait
`rl_parity_mid`	154.0	7.9	140	169	12.4
`rl_agent_mid`	157.1	9.3	135	176	13.8
`wilma`	159.0	10.9	138	181	12.9
`steffen`	159.1	10.1	142	184	13.0
`outside_in`	161.0	11.7	143	183	14.7
`rl_agent`	164.8	9.1	143	182	14.4
`random`	221.5	18.7	178	250	17.7
`back_to_front`	265.3	15.4	237	294	24.9

Performance Analysis on Target Domain

- **Agent Supremacy:** `rl_parity_mid` achieves **154.0 ticks**, surpassing the best heuristic (Steffen at 159.1 ticks and WILMA at 159.0 ticks).
- **Impact of Parity:** Adding row parity improved performance from 157.1 ticks (`rl_agent_mid`) to 154.0 ticks, providing the essential capability to express Steffen alternation.
- **Failure of Back-to-Front:** Back-to-Front required 265.3 ticks — 72% slower than the RL policy — proving that traditional airline boarding creates severe congestion.

8. Generalization Gap & Negative Results

Airbus A320 Out-of-Distribution Test (30 Rows, 180 Passengers). Lower is better.

Strategy	Mean Ticks	Std Dev	Best Seed	Worst Seed	Avg Wait
`steffen`	379.3	13.8	349	406	26.4
`wilma`	383.7	13.7	355	419	26.7
`outside_in`	386.9	12.7	362	421	31.4
`rl_agent_mid`	392.1	15.7	363	430	32.3
`rl_parity_mid`	394.0	13.6	371	434	28.8
`rl_agent`	407.0	14.1	369	435	34.0
`random`	493.5	26.7	433	559	33.3
`back_to_front`	615.6	30.5	566	691	52.7

The Generalization Gap Explained

On A320, the agent (394.0 ticks) is outperformed by Steffen (379.3 ticks) because quartiles span 7.5 rows instead of 2.5 rows, diluting spatial precision.

Autopsy of Other Negative Results (Preserved as Evidence)

Method Attempted	Observed Failure Mode	Root Cause Analysis
Linear Value FA	Severe divergence and oscillatory policy cycling.	Long horizon (>150 steps) coupled with binary seat states broke Bellman contraction.
Dense Congestion Shaping	Reward hacking: agent withheld passengers to avoid congestion penalties.	Shaped objectives deviated from pure makespan minimization, degrading overall boarding time.
Watkins' Q(λ) / SARSA(λ)	Failed to converge; plateaued at 195 ticks on mid cabin.	Bootstrapping across uniform –1 rewards failed to assign credit for delayed stowing bottlenecks.
Deep RL (DQN / PPO)	High variance, hyperparameter instability, sample inefficiency.	The compact 289-state discrete space does not warrant non-linear approximation; tabular is optimal.

Scientific Integrity Note: Both undertrained A320 direct policies and divergent linear FA models were permanently retired from the production registry rather than selectively showcased.

9. Software Engineering & API Architecture

The backend is constructed with FastAPI, enforcing schema validation via Pydantic and clean separation of concerns.

API Endpoint	HTTP	Functional Purpose in Architecture
`/api/health`	GET	Health check and service status.
`/api/options`	GET	Dynamically advertises valid aircraft presets and available policies.
`/api/simulate`	POST	Executes an episode; returns summary stats and full frame deltas for replay.
`/api/compare`	POST	Executes multiple strategies across a seed range, computing distributions.
`/api/train`	POST	Dispatches an asynchronous background training job.
`/api/train/{job_id}`	GET	Polls progress, reward curves, and status of active training jobs.
`/api/policies`	GET	Lists all serialized, verified policy files in the registry.

Structured Error Envelopes & API Pruning

Runtime exceptions return structured JSON: `{"error": {"code": "...", "message": "..."}}`. Incompatible options are pruned at schema layer.

10. Verification, Test Suite & Correctness Guarantees

Suite File	Test Count	Functional Areas Covered
`tests/test_agents.py`	23	State observation encoding, 24-action masking, reward signs, terminal states, policy serialization.
`tests/test_api.py`	12	FastAPI endpoint validation, error payloads, delta-frame serialization, options advertisement.
`tests/test_simulator.py`	31	Movement physics, aisle blocking, overhead luggage delays, seat shuffles, completion invariant.
Total Python Tests	**66 tests**	**100% passing automated test suite (pytest -q)**

The Hand-Verifiable Unit Test: Deterministic 4-row cabin (`tiny`) where expected boarding times are derived analytically by hand. Prevents arithmetic drift.

11. Deployment, Local Execution & Demonstration Protocol

```
# Terminal 1 — Backend API
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
PYTHONPATH=. .venv/bin/uvicorn api.main:app --reload --port 8000

# Terminal 2 — Frontend UI Cockpit
cd frontend && npm install && npm run dev     # open http://localhost:5173
```

Demonstration Protocol Exercising All Capabilities

- **1. Baseline vs Policy:** Select `mid`, seed 500, compare `steffen` (159.1 ticks) with `rl_parity_mid` (154.0 ticks).
- **2. Frame Scrubbing:** Scrub timeline back and forth with zero re-simulation overhead via delta frame arrays.
- **3. Compare & Generalization:** Run seeds 500–529 across all baselines on mid vs A320 to witness generalization gap.
- **4. Live Training:** Trigger 500-episode background job to inspect live reward curves in real time.

12. Technical Defense Q&A; & Interview Preparation

Is the agent genuinely learning, or merely reproducing a heuristic?

It is learning a state-conditioned release rule. The preference table contains 289 visited states with distinct probability distributions; it is not a lookup table for one of the five baselines. It beats both Steffen and WILMA on the training cabin while performing worse off-distribution.

Why is the reward function strictly −1 per tick with no shaping?

Because minimizing total ticks is the literal mathematical objective. Introducing intermediate heuristics risks reward hacking and alters the optimal policy. The pure makespan formulation guarantees theoretical alignment.

Why is the discount factor $\gamma = 1.0$?

The boarding process has a finite, guaranteed absorbing terminal state. Applying $\gamma < 1$ exponentially discounts delayed congestion penalties occurring 30 ticks later, actively penalizing the agent for planning ahead.

How do you prove the agent did not overfit the test set?

We maintained three strictly disjoint random seed sets: Training (1–48), Validation for restart selection (300–319), and Evaluation (500–529). The test set was never accessed during gradient updates or checkpoint selection.

Why does the policy lose to Steffen on the A320?

The action space abstracts row positions into 4 distance bands. A band spans 2.5 rows on the mid cabin but 7.5 rows on the A320. Quadrupling the training episodes on the A320 recovered only 5.5 ticks, confirming this is a structural state representation bottleneck.

Why use Tabular RL rather than Deep Neural Networks (DQN / PPO)?

The compressed state space has only 289 visited states. A neural network would introduce gradient instability, catastrophic forgetting, hyperparameter sensitivity, and non-determinism without addressing the primary bottleneck (sparse delayed credit assignment).

What is the primary operational limitation of this model?

Passengers are modeled as fully compliant cellular automata that board instantly when called. Real-world airline boarding involves non-compliant passengers, group travel (families refusing to separate), and gate crowding.

13. Conclusion & Future Roadmap

- **Core Contributions:** Deterministic CA simulator with 7 cabins; action-space row parity (12→24 actions) discovering alternating boarding; per-seed paired baseline REINFORCE achieving 154.0 ticks; honest documentation of A320 generalization bounds.
- **Future Directions:** Graph Neural Networks for geometry-invariant transfer across arbitrary cabin lengths; stochastic passenger compliance and family groups; dual-door apron bus coordination.

Report Submission Sign-off: Prepared and submitted for course *Reinforcement Learning* by Yash Garg (I023), Anumay Pandey (I054), and Yash Kothari (I037). All simulation code, agent training scripts, API servers, and test suites are maintained under version control in repository `yashgarg195/learning-to-board`.